

HACKBACK · REVERSE HACKATHON

Sentinel

Intrusion detection and auto-ban for a login page.

Stop the attacker. Spare the bystander. Catch the lucky guess.

Team **Hack Matrix** · Team ID **DBUG 770**

Card **SENTINEL** — Intrusion Detection & Auto-ban

Original **CrowdSec** · Multiplier **Brutal** ×1.2

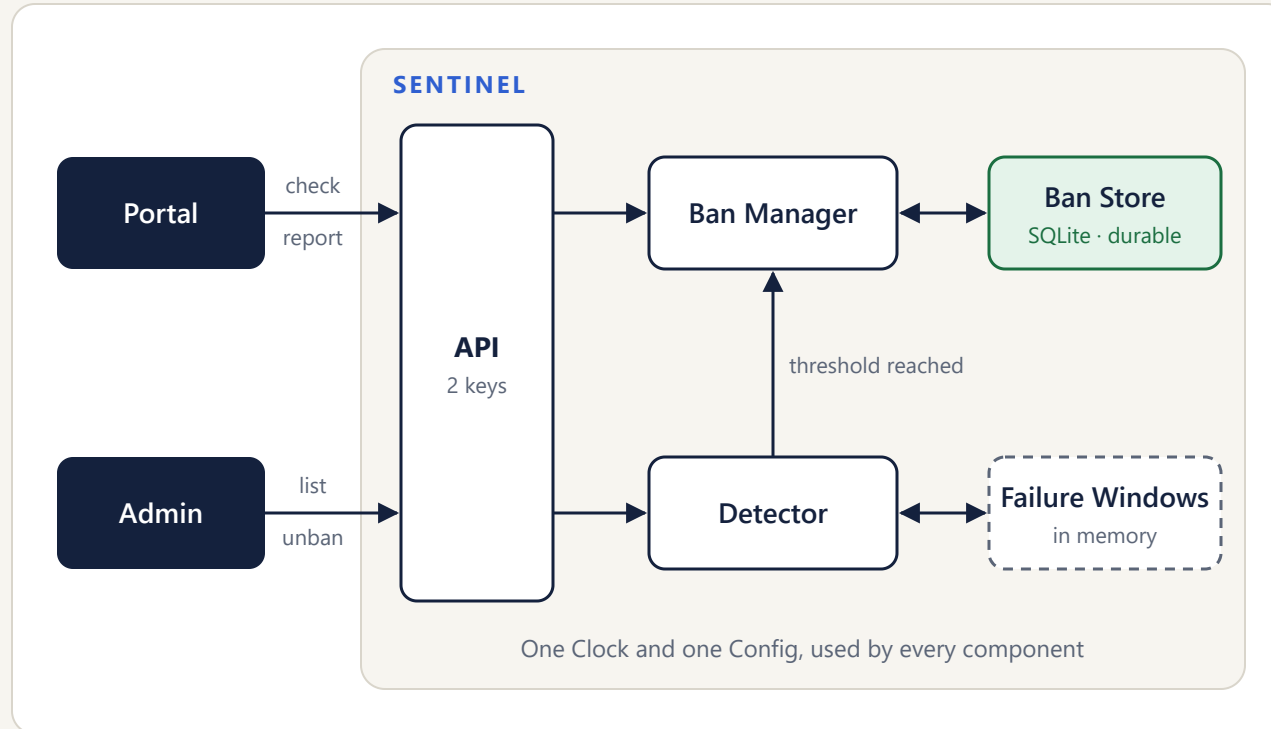
Clean-room rebuild · Python standard library + SQLite

THE PROBLEM

Bots guess passwords against a university portal, thousands of attempts from a few addresses.

- 1 Notice an address failing logins at attack speed.
- 2 Block it quickly and automatically.
- 3 Leave everyone else alone.
- 4 Let it back in at a known, exact time.

The behavior on the card, as one small service the portal calls directly



- **Events by API call.** The portal reports each login; no log parsing.
- **Sliding window.** 10 failures within 60 seconds; the 10th triggers the ban.
- **State computed when asked.** Active iff `now < expires_at`. No timer lifts a ban.
- **Per-IP atomicity.** Concurrent reports create exactly one ban and lose no failure.
- **Bans durable, counts in memory.** Bans survive a restart.

Four endpoints: `POST /v1/login-attempts`, `GET /v1/check`, `GET /v1/bans`, `DELETE /v1/bans/{id}`. Built only from our seven specification documents; no CrowdSec code or packages.

The three Killer Tests, automated through the HTTP API

KT-1

10 failed logins from one IP within a minute → banned

HOW

Failures are counted per IP in a 60-second sliding window. The ban exists before the 10th report is answered.

ASSERTED

Reports 1–9: **not banned**

Report 10: **banned** ban_triggered

KT-2

A normal user at the same time is not affected

HOW

Counts and bans are keyed by exact IP. A successful login is never counted.

ASSERTED

IP B before, during and after the attack on IP A: **allow: true**

KT-3

The ban is lifted exactly when it expires

HOW

Every check compares the clock to expires_at. Nothing has to run at expiry.

ASSERTED

expires_at - 1 ms: **banned**

expires_at: **not banned**

Boundaries are tested too: ten failures spanning exactly 60.000 s ban, spanning 60.001 s do not; reports from a banned IP never extend or retrigger the ban; 40 simultaneous failures create exactly one ban.

Less collateral damage, and a blocked person who knows why

IMPROVEMENT 1 · THE FIX

Existing sessions survive an IP ban

Many legitimate users share one public IP. A ban blocks new logins, not people already signed in.

GET /V1/CHECK	IP NOT BANNED	IP BANNED
context=login	allow	refuse
context=session	allow	allow

- A session check never changes the ban.
- No bypass: a login report from a banned IP is answered "banned", even with the correct password.
- Sentinel stores no session data.

IMPROVEMENT 2 · THE DIFFERENTIATOR

End-user ban explanation

Every "banned" answer carries an explanation the portal can show as it is.

Sign-in is temporarily blocked

Sign-in from your network address is temporarily blocked because of too many failed sign-in attempts.

Blocked since 2026-10-05T10:00:41.250Z
Blocked until 2026-10-05T10:15:41.250Z
Try again in 852 seconds

- Derived from the ban on each request; never stored.
- No usernames, evidence or rule values.
- null when not banned, including at the instant of expiry.

POST-SPEC EXTENSION

Catch the lucky guess

5 failed attempts → correct password → login succeeds → security alert

Login succeeds, but Sentinel still raises a security alert. Advisory: nothing is banned or counted.

POST-SPEC EXTENSION · OPTIONAL

AI incident brief

Sentinel's facts → Groq (optional) → short admin summary

Deterministic summary when Groq is absent or fails. The AI never makes a security decision.

Added after the specification, beyond the two required improvements; no claim about the original for either. *Stop the attacker. Spare the bystander. Catch the lucky guess.*

Shown live, and checked by 137 automated tests

137

tests, all passing

```
python -m unittest
```

Killer Tests	3
Core behavior	29
API contract	21
Improvement 1	8
Improvement 2	10
Success alert (extension)	27
AI brief (extension)	33
Demo portal	6

Live demo: a mock portal, one page

- 1 A student signs in on IP A.
- 2 The attacker fails 5 times, then guesses correctly: signed in, but flagged.
- 3 5 more failures: the 10th triggers the ban.
- 4 A user on IP B signs in, unaffected.
- 5 The student's existing session keeps working.
- 6 A new login on IP A is refused, with the explanation.
- 7 The countdown ends; IP A is unblocked at expires_at.
- 8 An admin lists bans, generates an incident brief, and unbans.

The demo is a separate client of the API. Sentinel has no UI.

Why this rebuild fits the card

- **The rule is the requirement.** "10 failures in 60 seconds" is stated directly, with defined boundaries.
- **One expiry rule everywhere.** Banned before expires_at, not banned at it.
- **Check built in.** The ban is in effect before the triggering report is answered.
- **One active ban per IP.**
- **No dependencies.** Python standard library and one SQLite file.

Comparison with the original is limited to what we observed by reading one commit; see docs/OBSERVATIONS.md and docs/GAPS.md.